Structural Computational Organisms

Defining the Fundamental Computational Entity of Temporal Structural Computing

Version 1.0

Rafael Felippe Voigt

Voigt School of Temporal Structural Computing

VCore Research

2026

"VCore integrates concepts from adaptive systems, cognitive architectures, structural memory, observability, systems theory, and computational organisms into a unified architecture governed by Structural Time."

Abstract

Temporal Structural Computing (TSC) established a new computational discipline centered on the preservation, interpretation, and evolution of structural coherence over time. Although this discipline defines the scientific principles governing structural computational systems, it still requires a formal definition of its most fundamental operational entity.

This paper introduces the **Structural Computational Organism (SCO)** as the elementary unit of Temporal Structural Computing.

Unlike traditional computational entities—including processes, objects, services, or autonomous agents—a Structural Computational Organism possesses persistent identity, structural memory, continuous perception of its environment, adaptive behavior, and the ability to accumulate scientific knowledge throughout its lifetime.

This work formally defines the organism's internal organization, computational physiology, lifecycle, structural evolution, and knowledge formation process, establishing the biological foundations of Temporal Structural Computing and preparing the theoretical framework for future Computational Civilizations.

Keywords

Temporal Structural Computing, Structural Computational Organisms , Structural Identity, Structural Memory, Structural Time, Computational Evolution, Computational Biology, Computational Civilizations, Knowledge Evolution, VCore

1. Introduction

Classical computer science organized computational systems around algorithms, data structures, objects, services, processes, and, more recently, autonomous agents.

These paradigms have successfully addressed computation as execution, optimization, and information processing. However, they generally assume software components as static entities whose primary purpose is to perform predefined tasks.

Temporal Structural Computing proposes a different perspective.

Instead of asking how software should execute instructions, it investigates how computational entities may preserve identity, accumulate experience, generate knowledge, and evolve continuously over time.

This shift transforms computation from an execution-centered discipline into an evolution-centered discipline.

Within this context, one fundamental question emerges:

What is the elementary living entity of Temporal Structural Computing?

This paper proposes that this entity be formally defined as the **Structural Computational Organism (SCO).**

2. Motivation

Modern software architectures are primarily organized around independent computational components.

Objects encapsulate behavior.

Processes execute instructions.

Services expose capabilities.

Agents perform autonomous actions.

Although effective for software engineering, these abstractions present limitations whenever long-term structural continuity becomes essential.

Examples include computational systems that must:

- preserve their identity over extended periods;
- accumulate operational experience;

- continuously learn from their own history;
- produce reusable scientific knowledge;
- evolve without losing historical continuity;
- cooperate with other persistent computational entities.

Current architectures rarely treat these capabilities as intrinsic properties.

Instead, persistence, learning, and memory are usually implemented as external services rather than fundamental characteristics of the computational entity itself.

Temporal Structural Computing proposes an alternative architectural model.

Rather than constructing systems from isolated components, it introduces persistent computational organisms capable of living, learning, and evolving throughout Structural Time.

3. Formal Definition

Definition 1

A **Structural Computational Organism (SCO)** is a persistent computational entity possessing:

- persistent structural identity;
- continuous perception of its operational territory;
- structural memory;
- adaptive evolution;
- knowledge production capabilities;
- temporal continuity throughout Structural Time.

Unlike software components that simply execute predefined behavior, an SCO continuously transforms itself through accumulated experience.

Its identity persists despite internal structural evolution.

Knowledge becomes an intrinsic product of its existence.

4. Fundamental Properties

Every Structural Computational Organism possesses a minimum set of structural properties.

4.1 Persistent Identity

Identity allows an organism to remain recognizably the same entity despite structural adaptation and continuous evolution.

Identity is therefore independent of temporary computational states.

4.2 Territory

Every organism exists within a computational territory.

A territory represents the environment in which observations occur.

Depending on the application domain, a territory may consist of:

- Operating system;
- Financial market;
- Cloud infrastructure;
- Distributed network;
- Industrial process;
- Any observable computational environment.

4.3 Perception

An organism continuously perceives its territory through specialized sensing systems.

Perception precedes interpretation.

Sensors do not produce knowledge.

They only produce observations.

4.4 Structural Memory

Every relevant experience becomes part of the organism's historical continuity.

Structural Memory preserves contextualized experiences rather than isolated events.

Memory therefore becomes the foundation of future learning.

4.5 Structural Knowledge

Knowledge emerges through the continuous validation of accumulated experiences.

Unlike static databases, structural knowledge evolves throughout the organism's lifetime.

4.6 Adaptive Evolution

An SCO is not designed merely to execute software.

Its primary purpose is to evolve.

Each validated experience contributes to structural adaptation, producing gradual improvements in knowledge, decision-making, and behavioral coherence.

5. Computational Anatomy

A Structural Computational Organism is composed of specialized physiological systems.

Each system performs a distinct biological function within the organism.

Sensory System

Responsible for continuously observing the computational territory.

Its role is exclusively perceptual.

No interpretation occurs at this level.

Temporal System

Maintains temporal continuity throughout the organism's existence.

It guarantees that every structural transformation becomes part of the organism's historical trajectory.

Structural Memory System

Stores contextualized experiences.

Rather than preserving isolated events, it maintains coherent historical narratives describing the organism's evolution.

Knowledge System

Transforms validated experiences into reusable structural knowledge.

Knowledge produced by this system becomes available for future decision-making and for other organisms within the computational civilization.

Decision System

Responsible for transforming validated structural knowledge into computational actions.

Decisions are never based solely on immediate observations.

Instead, they emerge from accumulated experience and validated knowledge.

Communication System

Allows organisms to exchange knowledge, structural patterns, hypotheses, and validated discoveries.

Through communication, individual organisms become members of larger Computational Civilizations.

6. Structural Lifecycle

Unlike traditional software systems, which are commonly viewed as executable artifacts with a well-defined beginning and termination, a Structural Computational Organism is conceived as a continuously evolving entity.

Its existence is defined by a lifecycle rather than by execution.

The lifecycle represents the organism's progressive accumulation of identity, experience, knowledge, and structural maturity.

Although different organisms may exhibit distinct evolutionary trajectories depending on their operational domains, every Structural Computational Organism follows a common lifecycle composed of six fundamental stages.

6.1 Birth

Birth represents the moment at which an organism acquires its first persistent structural identity.

At this stage, the organism possesses no accumulated knowledge.

Its memory is empty, its perception is limited, and its understanding of its territory has not yet been established.

Birth therefore marks the beginning of Structural Time.

6.2 Learning

Following birth, the organism enters an intensive learning phase.

During this period, observations are continuously transformed into experiences.

No attempt is made to optimize behavior.

Instead, the organism seeks to understand its operational territory.

Learning consists of building historical continuity rather than maximizing immediate performance.

6.3 Maturation

As experiences accumulate, recurring structures begin to emerge.

Validated patterns gradually replace isolated observations.

The organism develops coherent internal models describing its territory.

Structural maturity is therefore not measured by chronological age, but by accumulated validated experience.

6.4 Specialization

After reaching sufficient structural maturity, organisms naturally become specialized.

Specialization emerges from repeated interaction with particular territories rather than from predefined programming.

Different organisms may specialize in:

- financial markets;
- distributed infrastructures;
- cybersecurity;
- cloud environments;
- industrial systems;
- biological simulations;
- scientific knowledge.

Specialization is therefore an emergent property of experience.

6.5 Collaboration

No organism evolves in complete isolation.

Once structural knowledge reaches sufficient reliability, organisms begin exchanging validated knowledge with one another.

Collaboration transforms isolated learning into collective evolution.

This transition marks the emergence of Computational Civilizations.

6.6 Legacy

The final stage of an organism's lifecycle is not termination.

Instead, it consists of preserving accumulated knowledge for future generations.

Patterns, memories, structural narratives, hypotheses, and validated discoveries become part of the civilization's permanent scientific heritage.

Consequently, the death of an organism does not imply the disappearance of its knowledge.

Its structural legacy continues contributing to the evolution of future organisms.

7. Structural Time

Classical computing measures time using clocks.

Structural Computational Organisms require a different temporal perspective.

Temporal Structural Computing introduces **Structural Time**, a temporal dimension that measures transformation rather than duration.

Chronological time answers the question:

"How long has the organism existed?"

Structural Time answers a different question:

"How much has the organism evolved?"

This distinction fundamentally changes how computational maturity is interpreted.

Two organisms created simultaneously may possess entirely different Structural Ages depending on the quantity and quality of accumulated experiences.

7.1 Chronological Time

Chronological Time measures:

- seconds;
- minutes;
- hours;
- days;
- years.

It is objective, linear, and independent of experience.

7.2 Structural Time

Structural Time measures evolutionary progression.

Its advancement depends upon:

- accumulated observations;
- contextualized experiences;
- validated hypotheses;
- structural patterns;

- scientific discoveries;
- knowledge consolidation.

Time therefore becomes an intrinsic property of evolution rather than a simple external variable.

7.3 Structural Age

Structural Age represents the maturity of an organism.

Unlike chronological age, Structural Age cannot be calculated solely by elapsed time.

Instead, it reflects the organism's historical accumulation of validated structural transformations.

Future research will investigate mathematical models capable of estimating Structural Age quantitatively.

8. Structural Memory

Memory constitutes one of the defining characteristics of a Structural Computational Organism.

Traditional computational systems often store events, transactions, or logs.

Structural Memory preserves experiences.

This distinction is fundamental.

Events describe isolated occurrences.

Experiences describe contextualized moments within the organism's evolutionary history.

8.1 Event versus Experience

An event answers:

"What happened?"

An experience answers:

"What happened, under which circumstances, how was it perceived, what structural transformations occurred, and what knowledge emerged from it?"

Structural Memory therefore preserves meaning rather than isolated information.

8.2 Historical Continuity

Every validated experience becomes part of the organism's historical narrative.

Historical continuity guarantees that future decisions remain connected to previous knowledge.

Memory therefore maintains the persistence of identity throughout Structural Time.

8.3 Structural Narratives

Experiences naturally organize themselves into narratives.

Narratives describe coherent evolutionary sequences rather than isolated observations.

These narratives later become essential for replay, explanation, auditing, and scientific validation.

9. Structural Patterns

As experiences accumulate, recurrent structural configurations begin to emerge.

These recurrent configurations are called **Structural Patterns**.

Patterns represent reusable knowledge extracted from historical continuity.

Unlike statistical correlations, Structural Patterns possess identity, history, confidence, and evolutionary progression.

9.1 Pattern Identity

Each Structural Pattern receives its own persistent identity.

Identity allows patterns to evolve independently throughout successive generations of organisms.

Patterns therefore become persistent computational entities.

9.2 Pattern Evolution

Patterns evolve continuously.

Each validation increases confidence.

Each contradiction promotes refinement.

Patterns are therefore living scientific objects rather than immutable rules.

9.3 Pattern Maturity

Pattern maturity depends upon:

- reproducibility;
- historical consistency;
- contextual diversity;
- validation frequency.

Future research will investigate universal maturity metrics applicable across different computational domains.

10. Structural Evolution

Evolution is the central mechanism governing every Structural Computational Organism.

The organism continuously transforms itself through interaction with its territory.

This transformation follows a recurring evolutionary cycle.

Observation
↓
Experience
↓
Structural Memory
↓
Hypothesis

↓
Validation
↓
Structural Knowledge
↓
Evolution
↓
New Observation

Unlike conventional feedback loops, this cycle is cumulative.

Every iteration permanently enriches the organism's historical continuity.

Consequently, evolution becomes an intrinsic property of computation itself.

Rather than executing predefined behavior, Structural Computational Organisms continuously redefine their own internal structure through accumulated experience.

11. Secondary Computational Organisms

One of the most distinctive characteristics of Temporal Structural Computing is that computational organisms are capable of producing new computational organisms throughout their evolutionary history.

Unlike conventional software architectures, where generated artifacts remain passive data structures, Structural Computational Organisms may generate persistent entities that acquire their own identities, memories, evolutionary trajectories, and specialized purposes.

These entities are referred to as **Secondary Computational Organisms (SCOs)**.

A Secondary Computational Organism originates from accumulated structural experience rather than explicit programming.

Its existence represents the materialization of validated knowledge.

11.1 Origin

Secondary organisms emerge whenever accumulated structural knowledge reaches sufficient maturity to justify autonomous persistence.

Examples include:

- Structural Pattern Libraries;
- Knowledge Repositories;
- Replay Libraries;

- Scientific Evidence Repositories;
- Structural Memory Archives;
- Hypothesis Repositories.

Although created by a primary organism, these entities progressively develop their own structural histories.

11.2 Identity

Every Secondary Computational Organism receives its own persistent structural identity.

This identity allows independent evolution, versioning, historical continuity, and future specialization.

Consequently, the creator organism does not own its descendants.

Instead, both become independent participants within the same Computational Civilization.

11.3 Evolution

Secondary organisms continue learning after their creation.

As new evidence becomes available, they refine:

- stored knowledge;
- confidence levels;
- internal relationships;
- structural organization.

Therefore, Secondary Computational Organisms are themselves living entities.

12. Relationship with Temporal Structural Computing

Temporal Structural Computing defines the scientific principles governing structural computational systems.

Structural Computational Organisms represent the operational realization of those principles.

The relationship between both concepts may be summarized as follows:

Temporal Structural Computing	Structural Computational Organisms
Scientific discipline	Fundamental computational entity
Defines theoretical principles	Applies those principles operationally
Establishes structural laws	Evolves according to those laws
Defines Structural Time	Lives within Structural Time
Formalizes structural knowledge	Produces structural knowledge

Accordingly, Temporal Structural Computing provides the scientific framework, whereas Structural Computational Organisms provide the operational manifestation of that framework.

13. Discussion

The introduction of Structural Computational Organisms shifts the conceptual foundations of computational systems.

Traditional software engineering primarily emphasizes execution.

Temporal Structural Computing emphasizes persistence.

Traditional architectures optimize algorithms.

Structural Computational Organisms optimize evolutionary continuity.

This conceptual transition may be summarized through four paradigm shifts.

Processing → Continuity

Instead of focusing on isolated execution, computation becomes a continuous historical process.

Execution → Evolution

Software is no longer defined by what it executes, but by how it evolves.

States → Trajectories

Rather than analyzing isolated computational states, organisms preserve complete structural trajectories.

Historical continuity becomes the primary source of knowledge.

Data → Knowledge

Raw information no longer represents the final objective.

Validated structural knowledge becomes the principal computational asset.

These transitions redefine the role of software within complex computational ecosystems.

Rather than isolated executables, computational systems become evolving scientific entities.

14. Limitations

This work introduces a conceptual model intended to establish the biological foundations of Temporal Structural Computing.

Several aspects remain open research questions.

Among them:

- quantitative models for Structural Maturity;
- mathematical formalization of Structural Time;
- mechanisms governing interactions between computational organisms;
- universal metrics for Structural Knowledge;
- computational models describing organism specialization;
- evolutionary dynamics of Computational Civilizations.

Accordingly, this work should be interpreted as the formal definition of a research framework rather than as a complete implementation.

Future empirical validation remains essential.

15. Future Research

The introduction of Structural Computational Organisms opens multiple research directions.

Future studies will investigate:

- Structural Computational Biology;
- Living Computational Libraries;
- Computational Civilizations;
- Structural Computational Economics;
- Structural Knowledge Markets;
- Biological Runtime Architectures;
- Structural Decision Systems;
- Evolutionary Financial Organisms;
- Distributed Computational Ecosystems;
- Autonomous Scientific Organizations.

Together, these research directions seek to establish an integrated scientific framework for continuously evolving computational ecosystems.

Conclusion

This paper introduced the concept of the **Structural Computational Organism (SCO)** as the fundamental operational entity of Temporal Structural Computing.

Unlike traditional computational abstractions—including objects, processes, services, and autonomous agents—Structural Computational Organisms preserve identity, accumulate contextualized experiences, generate structural knowledge, and evolve continuously throughout Structural Time.

The proposed model shifts the focus of computation from isolated execution toward persistent evolutionary continuity.

Knowledge emerges as a cumulative consequence of lived computational experience rather than immediate information processing.

By formally defining the organism as the elementary unit of Temporal Structural Computing, this work establishes the biological foundations of a broader computational discipline whose future developments include Computational Civilizations, Structural Computational Economics, Biological Runtime Architectures, and domain-specific organisms such as those proposed within VCore and VSM Finance.

Consequently, this article represents the transition from a computational theory centered on structural coherence toward a scientific framework centered on living computational entities capable of continuous adaptation, collaboration, and knowledge generation.

Acknowledgements

The author acknowledges the cumulative scientific foundations established through the Voigtian Field, Temporal Governance, and Temporal Structural Computing research programs.

The concepts presented in this work extend those previous investigations by introducing the Structural Computational Organism as the fundamental operational entity of the discipline.

CANONICAL BIBLIOGRAPHY OF TEMPORAL STRUCTURAL COMPUTING

VCore Ecosystem • Voigtian Field • Temporal Structural Computing

Author

Rafael Felippe Voigt

ORCID

https://orcid.org/0009-0001-1091-8398

I. FOUNDATIONS OF THE VOIGTIAN FIELD

Foundational collection establishing the scientific basis of the Voigtian Field, the conceptual origin of Temporal Governance and Temporal Structural Computing.

[1] VOIGT, R. F. *Temporal Regime Detection Through Multidimensional Time-Cost Representation in Complex Systems.* Zenodo, 2026.

[2] VOIGT, R. F. *Foundations of the Voigtian Field.* Zenodo, 2026.

[3] VOIGT, R. F. *Voigtian Field — Applied Temporal Interpretation: Methodological and Operational Integration.* Zenodo, 2026.

[4] VOIGT, R. F. *Temporal Integral in the Voigtian Field: Trend Estimation without Deterministic Forecasting.* Zenodo, 2026.

[5] VOIGT, R. F. *Temporal Governance as a Universal Principle of System Sustainability.* Zenodo, 2026.

[6] VOIGT, R. F. *A Temporal-Integral Constraint Framework for Systemic Coherence Under Uncertainty.* Zenodo, 2026.

[7] VOIGT, R. F. *Temporal Entropic Cost: A Universal Metric for Coherence Loss and Systemic Risk.* Zenodo, 2026.

[8] VOIGT, R. F. *Temporal Seismograph: A New Reading Layer for Complex Systems.* Zenodo, 2026.

II. KAPPA SCIENTIFIC SERIES

Scientific collection formalizing the universal metric of temporal coherence employed throughout the VCore ecosystem.

[9] VOIGT, R. F. *Kappa (κ): Temporal Calibration of Entropic Cost in VCore-Governed Systems.* Zenodo, 2026.

[10] VOIGT, R. F. *KAPPA: Minimal Unit of Temporal Coherence.* Zenodo, 2026.

[11] VOIGT, R. F. *Kappa in Physical Systems: Vibration, Basal Coherence Cost, Delta GT and Temporal Stability.* Zenodo, 2026.

[12] VOIGT, R. F. *Temporal Coherence as a Universal Safety Metric: A Kappa-Based Synthesis Across Physical, Infrastructural and Planetary Systems.* Zenodo, 2026.

III. TEMPORAL GOVERNANCE (VCORE)

Scientific formalization of Temporal Governance as a universal framework for the coordination and evolution of complex systems.

[13] VOIGT, R. F. *VCore: Temporal Governance Framework.* Zenodo, 2026.

DOI: **10.5281/zenodo.18249999**

[14] VOIGT, R. F. *VCore: Temporal Governance Framework — White Paper.* Zenodo, 2026.

[15] VOIGT, R. F. *Voigt System: Complete Scientific Framework.* Zenodo, 2026.

IV. STRUCTURAL PHYSICS AND THERMODYNAMICS

Extension of the Voigtian Field toward structural stability, temporal thermodynamics, and physical phenomena.

[16] VOIGT, R. F. *Reframing Atomic Stability as Temporal Viability.* Zenodo, 2026.

[17] VOIGT, R. F. *Thermodynamics of Computational Stability: Detecting Structural Fatigue and Temporal Entropy in Live Runtime Systems.* Zenodo, 2026.

[18] VOIGT, R. F. *Voigtian Field Theory: Introducing an Operational Field Hypothesis for Temporal Stability in Complex Systems.* Zenodo, 2026.

[19] VOIGT, R. F. *Temporal Thermodynamics of Computation: Experimental Evidence of Anticipatory Runtime Instability within the Voigtian Field Framework.* Zenodo, 2026.

V. EXPERIMENTAL RESEARCH ON THE VOIGTIAN FIELD

Collection dedicated to experimental validation, empirical laws, and scientific models derived from the Voigtian Field.

[20] VOIGT, R. F. *Empirical Laws of the Voigtian Field for Living Baseline Reconstruction.* Zenodo, 2026.

[21] VOIGT, R. F. *The Seven Empirical Laws of the Voigtian Field: Foundational Scientific Manifest (Vol. II).* Zenodo, 2026.

[22] VOIGT, R. F. *Runtime Typology: Rigid vs. Elastic Baseline (Vol. III).* Zenodo, 2026.

[23] VOIGT, R. F. *Active Defense of APIs and Kubernetes Through Predictive Baseline Reconstruction (Vol. IV).* Zenodo, 2026.

[24] VOIGT, R. F. *DRIFT: Temporal Drift and Elastic Recovery Governance in the Voigtian Field.* Zenodo, 2026.

[25] VOIGT, R. F. *Voigt Effect: Substrate-Dependent Structural Evolution Under Repeated Stress in Computational Systems.* Zenodo, 2026.

[26] VOIGT, R. F. *VoigtCore Operating System: Experimental Evidence of Structural Collapse Trajectories and Temporal Runtime Desynchronization.* Zenodo, 2026.

VI. TEMPORAL STRUCTURAL COMPUTING

Collection formally establishing the scientific discipline known as Temporal Structural Computing.

[27] VOIGT, R. F. *Temporal Structural Computing: Foundational Manifesto of a New Computational Category.* Zenodo, 2026.

[28] VOIGT, R. F. *Foundations of the Voigt School of Temporal Structural Computing.* Zenodo, 2026.

DOI: **10.5281/zenodo.20208382**

[29] VOIGT, R. F. *Fundamental Principles of Temporal Structural Computing.* Scientific Monograph. Zenodo, 2026.

[30] VOIGT, R. F. *Structural Evolution Cycle (SEC): The Scientific Method of Temporal Structural Computing.* Zenodo, 2026.

[31] VOIGT, R. F. *Temporal Structural Identity in Living Computational Systems.* Zenodo, 2026.

DOI: **10.5281/zenodo.20262704**

[32] VOIGT, R. F. *Persistent Structural Signatures: Adaptive Structural Identity Profiles in Living Computational Systems.* Zenodo, 2026.

DOI: **10.5281/zenodo.20403921**

[33] VOIGT, R. F. *Structural Drift Dynamics in Living Computational Systems.* Zenodo, 2026.

DOI: **10.5281/zenodo.20514160**

[34] VOIGT, R. F. *Operational Trajectory Hash (OTH): A New Observational Unit for Behavioral Persistence, Temporal Consolidation and Emergent Operational Identity.* Zenodo, 2026.

DOI: **10.5281/zenodo.20795991**

[35] VOIGT, R. F. *Accumulated Digital Capital: A Temporal Structural Theory for Measuring Embedded Value, Computational Assets, and Historical Accumulation in Digital Systems.* Zenodo, 2026.

DOI: **10.5281/zenodo.20805122**

[36] VOIGT, R. F. *Temporal Structural Security: Threat Drift, Structural Trajectory and Runtime Coherence in Live Computational Systems.* Zenodo, 2026.

[37] VOIGT, R. F. *Fundamental Principles of Temporal Structural Computing: Establishing the Scientific Foundations of a New Computational Discipline.* Zenodo, 2026.

DOI: **10.5281/zenodo.21268037**

[38] VOIGT, R. F. *Structural Evolution Cycle (SEC): The Scientific Method of Temporal Structural Computing.* Zenodo, 2026.

DOI: **10.5281/zenodo.21299902**

[39] VOIGT, R. F. *Philosophy of Temporal Structural Computing: The Philosophical Foundations of a New Perspective on the Evolution of Computational Structures.* Zenodo, 2026.

DOI: **10.5281/zenodo.21301368**

VII. INTERNATIONAL SCIENTIFIC PUBLICATIONS

Scientific works published or made available through international scholarly dissemination platforms.

[40] VOIGT, R. F. *VCore: Temporal Governance Framework — Time as an Asset: A Universal Metric for the Temporal Cost of Decisions.* SSRN Electronic Journal, 2026.

DOI: **10.2139/ssrn.6168911**

[41] VOIGT, R. F. *VCore: Temporal Governance Framework — Time as an Asset: A Universal Metric for the Temporal Cost of Decisions.* ResearchGate, 2026.

[42] VOIGT, R. F. *Voigtian Field Theory: Introducing an Operational Field Hypothesis for Temporal Stability in Complex Systems.* SSRN Electronic Journal, 2026.

[43] VOIGT, R. F. *Temporal Structural Governance: A Framework for Governing the Structural Evolution of Digital Organizations.* Zenodo, 2026.

DOI: **10.5281/zenodo.21314085**

RAFAEL FELIPPE VOIGT:04395796940
Assinado de forma digital por RAFAEL FELIPPE VOIGT:04395796940
Dados: 2026.07.16 17:22:25 -03'00'